HNGIN

An attempt to implement a 3D rendering engine based on mere intuition and my little knowledge of linear algebra.

HNGIN is an attempt to implement a three-dimensional rendering engine from scratch, based mainly on my intuition, my limited knowledge of linear algebra and some research and information gathering that I have carried out in order to achieve certain functionalities.

As can be inferred from the current state of the project, it should be treated for the moment as a proof of concept rather than a stable and secure product, since there are still many functionalities to be implemented that are necessary in any rendering engine (see the to-do section), as well as some possible bugs to cover, improve the code in many aspects and write documentation for all the features and operation of the software (however, for a greater understanding of this aspect one can go to the intuition behind the engine section); it is also necessary to mention the detail that this program is written entirely in Python, which makes it relatively slow in comparison, although somewhat easier to read.

However, and as a result of the relatively important progress that I have been achieving in this project, I have decided to make it public and freely accessible in order to facilitate its use and to be able to receive suggestions, feedback and contributions from interested users.

Run the demonstration program

▼ Section table of contents

1. Clone the repository
2. Getting the necessary dependencies.
3. Running the engine source code.
4. Packaging the program in an executable binary.

Clone the repository

First, one has to clone the code repository on its local machine using git, to do this, use the following command:

```
git clone https://github.com/h3nry-d1az/hngin.git
```

Once the repository is cloned, proceed to move to the recently cloned directory using the command shown below:

```
cd hngin/
```

Getting the necessary dependencies

In order to run the demo program, you must have the pygame library installed, which is responsible for managing two-dimensional graphics. If you do not have the aforementioned module installed on your system, you can obtain it with the following command:

```
pip install pygame
```

For more information, one can go to the [official website] of the said library.

Running the engine source code

Once the previous dependencies have been installed, one can finally run the engine demo program. To do this, first go to the root directory of this project and run the following command in your terminal:

```
python ./main.py
```

Which should cause two windows to be displayed on the screen, a rectangular one (640x480), centered on the screen (approximately) and a longer one located right next to it, which processes the engine configurations.

Packaging the program in an executable binary

To compile the source code of the HNGIN demo program to an executable binary, you will require the PyInstaller package on your machine, you can get it by running the following command:

```
pip install pyinstaller
```

Again, for more information you can go to the [project website]. Once you have the dependencies installed on your device, proceed to execute the command shown below to produce the desired binary:

```
pyinstaller --onefile --noconsole ./main.py
```

The project binaries will **NOT** be available in the releases section.

Project roadmap

Even though enough features have been implemented within the engine to have been able to make the decision to make it public, most of the functionalities that I would like to see covered and that I think would be feasible are not yet implemented. Therefore, below is the roadmap designed for the project (although without any date, it is more than likely that not all of them will end up being fulfilled):

- [x] Achieve orthogonal projection.
- [x] Achieve projection based on focal length and linear interpolation.
- [x] Importing .obj files (vertices).
- [x] Importing .obj files (faces).
- [x] Fix visual glitches.
- [x] Implement camera rotations.
- [x] Implement a settings bar.
- [ ] Change the use of edges for faces.
- [ ] Optimize code and improve execution speed.
- [ ] Add solid colors to the models' faces.
- [ ] Add support for textures in models.

(Almost) all progress on this project has been recorded and saved to the YouTube playlist "Motor 3D HNGIN". Any contribution that helps carry out the above goals will always be welcome.

Intuition behind the engine code

▼ Section table of contents

1. Introduction.
2. Modeling the vertices of an object.
3. Modeling faces as edges (vertex unions).
4. Projecting the vertices and edges.
5. A detail to keep in mind.
6. Camera rotations.
7. Works Cited.

Introduction

The idea behind HNGIN lies in, first, modeling three-dimensional objects through mathematical structures with which we can then operate (to, for example, move or rotate the said objects), and manipulate this three-dimensional space to project it into a two-dimensional one, since we can draw this on the screen easily using the pygame library mentioned above.

Modeling the vertices of an object

By its own definition, every vertex of a three-dimensional object consists of three coordinates: x, y and z [1, 2], thus we can model this vertex as a three-dimensional vector (of three components), linking this concept to the results of linear algebra and being able to operate with vertices of mathematical way. More rigorously, every vertex with coordinates x, y, z corresponds to one and only one vector $\mathbf{p} = [x, y, z]$ such that $\mathbf{p} \in \mathbb{R}^3$.

Modeling faces as edges (vertex unions)

As mentioned in resources [1] and [2], three-dimensional objects (at least those in .obj format) are also composed of faces (triangles), however, we can simplify these as three edges that join three points, then our job is reduced to modeling these edges. Given $\mathbf{p}$ and $\mathbf{q}$ vertices, we will define an edge $\overline{\mathbf{pq}}$ as the line segment that begins at $\mathbf{p}$ and ends at $\mathbf{q}$ [6], that is, that joins $\mathbf{p}$ with $\mathbf{q}$. Note that it is commutative, so $\overline{\mathbf{pq}} = \overline{\mathbf{qp}}$, since the said line segment is not oriented.

Projecting the vertices and edges

Once one has modeled the two fundamental concepts of three-dimensional objects (for the moment, since to date I do not know how to draw faces with solid colors nor work with normals) in a three-dimensional space, we will have to find a way to project it. so that we can see it on the screen, which is a two-dimensional surface.

Orthogonal projection

The first approach that one can take when facing this problem is to simply delete the z coordinate of each three-dimensional vector and render the resulting two-dimensional vector, or what is equivalent and more formal taking into account the context of linear algebra, apply a linear transformation that simply "squishes space" onto a 2D surface, which corresponds to the following vector-by-matrix multiplication:

$$\mathbf{p}' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \mathbf{p}$$

This procedure is called orthogonal projection [3], and although the intuition behind it is simple, the result, although clearly not perfect (I mean, the lack of depth and perspective is evident), is a valid starting point.

Projection based on focal length and linear interpolation

Another more accurate approach that we can take to this problem lies in the field of optics, and I must say that my knowledge in this branch of physics is constrained only to the limited research that I have done for the purpose of carrying out this project, so I quote verbatim the explanation behind the method I ended up using: "Let's assume you stand in front of a window, looking out. If you stand in the center of the window, looking out through the center of the window, then we can treat the center of your eye (more precisely, the center of the lens in the pupil of your dominant eye) the origin in 3D coordinates ... Let's say one of the 3D coordinates of an interesting detail, [any given point], are (x, y, z). That ray intersects the window at $x' = x \cdot \frac{f}{z}, y' = y \cdot \frac{f}{z}, z' = f$. Therefore, the 2D coordinates of that detail on the window are: $(x', y') = \left(x\frac{d}{z}, y\frac{d}{z}\right) = \frac{d}{z}(x, y)$" [4]. In this case, as is obvious, the window is our screen.

To try to summarize and simplify this paragraph, we compare our screen with a window through which we see the scene, and through linear interpolation we can work and show that a point (x, y, z) will have as coordinates in the window $(x \cdot \frac{f}{z}, y \cdot \frac{f}{z}, f)$, with f being our focal length [5], then to project a vertex $\mathbf{p}$ we will have to apply the following transformation (note that it is not linear, we cannot use matrices in this case):

$$\mathcal{P} : \begin{bmatrix} x \\ y \\ z \end{bmatrix} \mapsto \begin{bmatrix} x \cdot \frac{f}{z} \\ y \cdot \frac{f}{z} \\ f \end{bmatrix}$$

Since the window is our screen, we can delete the z coordinate and render the resulting two-dimensional vector, we can use the previously seen orthographic projection for this:

$$\mathbf{p}' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \mathcal{P}(\mathbf{p})$$

And the resulting two-dimensional vector $\mathbf{p}'$ is the point to be rendered on the screen.

A detail to keep in mind

One must keep in mind that any point with a z coordinate lower than that of the camera must not be rendered (nor the previous formulas applied), as it would cause graphic errors mainly when drawing the edges on the screen. I reiterate, with a z coordinate less than that of the camera, that is, less than zero, since we are working with the camera being fixed at the origin; if it could be moved, you would simply have to subtract the vector in question and the vector with the camera position:

$$\mathbf{p}_n = \mathbf{p} - \mathbf{c}$$

From this section on, when we refer to a vertex (vector) and its transformations, it must be normalized. Once the operations have been carried out, we can obtain the original vector again by adding the camera position vector to it:

$$\mathbf{p} = \mathbf{p}_n + \mathbf{c}$$

Camera rotations

The most trivial (although valid, and the one I ended up using) approach one can take when implementing camera rotations in a three-dimensional space is to simply rotate the rest of the stage in the opposite direction, that is, rotate the camera to the left involves rotating the scene to the right, or rotating it up involves rotating it down.

This, however, raises two not so obvious questions: first, how to rotate a three-dimensional vector properly (as we will see later, in what order to apply the rotation matrices), and also, that the aforementioned 3D space consists of three axes, that is, six possible directions of rotation, while (ideally, and as it works in most systems) we only have four keys, two for vertical movement and another two for horizontal movement.

Rotate a three-dimensional vector

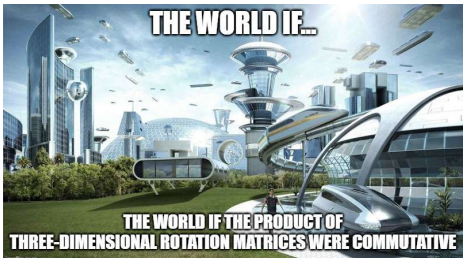
We will begin by resolving the first of the said questions. The space $\mathbb{R}^3$ has, as we have already mentioned, three coordinate axes: X, Y and Z, therefore we can consider three different axes of rotation when rotating any vector $\mathbf{p}$, that is, there are three different rotation matrices we can employ in order to revolve our given $\mathbf{p}$ (one for each axis) [7]:

$$R_X(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

$$R_Y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_Z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

As it is possible to infer, we need to rotate each vector around each of the three axes (with a rotation angle probably different for each of them), which implies applying each of these three matrices consecutively, or what is the same, apply the composition (product) of the three, that is, an even larger matrix.



The problem comes, however, that the product of these matrices is not commutative, that is, the order in which we apply these rotations matters, then rotate 27° around the X axis, then 84° around the Y and finally 11° around to Z, is not the same as 11° around Z, then 84° around Y and finally 27° with respect to X. So, in what order should we apply these matrices? Most of the resources I consulted suggested applying R_X first, then R_Y and finally R_Z [7, 8, 10] (I now understand that they were referring to another type of rotation, but not by then), except for one in particular that stated to first apply R_Y, then R_X and finally R_Z [9].

However, none of these methods worked, giving me wrong perspectives of the elements on the screen, which was because the rotation around the Z axis was not working.

Thinking about it, if we first rotate around the X and Y axes, any element that is on the X axis or, in general, in any position, and that we are looking at will be located right on the Z axis, then applying a rotation around the Z axis will make it rotate in the undesired way seen previously.

The solution? Get rid of the rotation around the Z axis as soon as possible, that is, rotate about the aforementioned axis first, and then apply the rotation in X and finally in Y. This method is the one that I ended up using and that, in truth, it works.

In general, for a vertex $\mathbf{p}$ and camera rotation angles θ_x, θ_y and θ_z, the three-dimensional vector resulting from the rotation process will be:

$$\mathbf{p}' = R_Y(-\theta_y)R_X(-\theta_x)R_Z(-\theta_z)\mathbf{p}$$

Which, computed, has the following appearance:

$$\begin{bmatrix} \mathbf{p}_z\sin(\theta_y)\cos(\theta_x) + \mathbf{p}_y(\sin(\theta_x)\sin(\theta_y)\cos(\theta_z) - \sin(\theta_z)\cos(\theta_y)) + \mathbf{p}_x(\sin(\theta_x)\sin(\theta_y)\sin(\theta_z) + \cos(\theta_y)\cos(\theta_z)) \\ -\mathbf{p}_z\sin(\theta_x) + \mathbf{p}_x\sin(\theta_z)\cos(\theta_x) + \mathbf{p}_y\cos(\theta_x)\cos(\theta_z) \\ \mathbf{p}_x(\sin(\theta_x)\sin(\theta_z)\cos(\theta_y) - \sin(\theta_y)\cos(\theta_z)) + \mathbf{p}_y(\sin(\theta_x)\cos(\theta_y)\cos(\theta_z) + \sin(\theta_y)\sin(\theta_z)) + \mathbf{p}_z\cos(\theta_x)\cos(\theta_y) \end{bmatrix}$$

And finally projected on screen:

$$\mathbf{p}'' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \mathcal{P}\left(\begin{bmatrix} \mathbf{p}_z\sin(\theta_y)\cos(\theta_x) + \mathbf{p}_y(\sin(\theta_x)\sin(\theta_y)\cos(\theta_z) - \sin(\theta_z)\cos(\theta_y)) + \mathbf{p}_x(\sin(\theta_x)\sin(\theta_y)\sin(\theta_z) + \cos(\theta_y)\cos(\theta_z)) \\ -\mathbf{p}_z\sin(\theta_x) + \mathbf{p}_x\sin(\theta_z)\cos(\theta_x) + \mathbf{p}_y\cos(\theta_x)\cos(\theta_z) \\ \mathbf{p}_x(\sin(\theta_x)\sin(\theta_z)\cos(\theta_y) - \sin(\theta_y)\cos(\theta_z)) + \mathbf{p}_y(\sin(\theta_x)\cos(\theta_y)\cos(\theta_z) + \sin(\theta_y)\sin(\theta_z)) + \mathbf{p}_z\cos(\theta_x)\cos(\theta_y) \end{bmatrix}\right)$$

Camera rotation controls

Once we can rotate the camera properly, we have to simplify the necessary controls to only depend on four keys instead of six, that is, two that control the vertical rotation and two that control the horizontal one, instead of two for each axis.

The horizontal rotation is simple, because it remains the same as the rotation around the Y axis, since it is independent of the others in the sense that, no matter the position of the camera, it will always work properly. We can assign two keys then to increment and decrement the θ_y angle.

Vertical rotation, however, is more complicated, since it is necessary to rotate around the X and/or Z axis, which depends on the value of θ_y. When the camera is at the origin without having made any rotation, we are observing along the Z axis, since this is the depth axis, while when we rotate our camera $\pm\frac{\pi}{2}$ radians, we are observing along the X axis. That an element is located on the Z axis requires a rotation around the X axis so that the effect of verticality can be appreciated, since a rotation around Z will produce the effect seen in the bad rotation example shown above, and vice versa happens with an element located in X, it will require a rotation around Z.

For now, we can create a special angle called λ which stores the total vertical rotation, later we will convert it to θ_x and θ_z. We will use two keys to increase and decrease the λ value.

Now, when θ_y is 0 and we rotate vertically, we increment θ_x completely and do not touch θ_z, while when θ_y takes the value of $\pm\frac{\pi}{2}$, θ_z increments completely and θ_x does not. For any angle θ_y, we can then generalize and define θ_x and θ_z in terms of θ_y and λ using trigonometry:

$$\theta_x = \lambda\cos(\theta_y)\ ;\ \theta_z = \lambda\sin(\theta_y)$$

Finally, to avoid graphical errors, we will keep the θ_x, θ_y and θ_z angles (θ_y and λ) always in the interval $[-\pi, \pi]$.

Works Cited

1. *Guidance to write a parser for .Obj and mtl file*, http://web.cse.ohio-state.edu/~shen.94/581/Site/Lab3_files/Labhelp_Obj_parser.htm. Accessed 3 October 2023.
2. *CS 418 - Parsing OBJ Files*, 17 April 2023, https://cs418.cs.illinois.edu/website/text/obj.html. Accessed 3 October 2023.
3. Margalit, Dan, and Joseph Rabinoff. *Orthogonal Projection*, https://textbooks.math.gatech.edu/ila/projections.html. Accessed 4 October 2023.
4. "3D projection on a 2D plane (weak maths ressources)." *Mathematics Stack Exchange*, 1 June 2017, https://math.stackexchange.com/questions/2305792/3d-projection-on-a-2d-plane-weak-maths-ressources. Accessed 4 October 2023.
5. "Focal Length of a Lens." *HyperPhysics Concepts*, http://hyperphysics.phy-astr.gsu.edu/hbase/geoopt/foclen.html. Accessed 4 October 2023.
6. *Euclid's Postulates*, https://people.math.harvard.edu/~ctm/home/text/class/harvard/113/97/html/euclid.html. Accessed 4 October 2023.
7. *3DRotations*, http://motion.pratt.duke.edu/RoboticSystems/3DRotations.html. Accessed 6 October 2023.
8. "Rotation matrix." *Wikipedia*, https://en.wikipedia.org/wiki/Rotation_matrix. Accessed 6 October 2023.

9. “How to Make a Three-Dimensional Project.” *Scratch Wiki*, 2 July 2023, https://en.scratch-wiki.info/wiki/How_to_Make_a_Three-Dimensional_Project#Rotation. Accessed 6 October 2023.
10. Plein, Dominic. “Extrinsic & intrinsic rotation: Do I multiply from right or left?” Dominic Plein, 2 May 2022, https://dominicplein.medium.com/extrinsic-intrinsic-rotation-do-i-multiply-from-right-or-left-357c38c1abfd. Accessed 6 October 2023.

License

MIT License

Copyright (c) 2023 Henry Díaz Bordón

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

> This project is dedicated mainly to my **parents**, who, despite being very far away, are still with me; but also to my **host family**, to whom I couldn't be more grateful for all the effort they are making.